PROJECT SUTRA

AI in the quotation workflow

Java orchestrates the job; Python reads file formats; Groq extracts fields; people check the result.

1. **Upload** — Vendor sends PDF, Excel or Word
2. **Validate** — Type, signature bytes, size, Office structure
3. **Parse** — Python FastAPI returns text and tables
4. **Extract** — Groq returns strict-schema JSON
5. **Recheck** — Java validates, matches and recalculates
6. **Review** — Buyer checks flags; manager decides

SCANNED PDF

PDFBox renders pages

Local Tesseract OCR reads the page image. OCR is used when text is missing or marked scanned.

GROQ API

Default model

openai/gpt-oss-120b

Fallback models: gpt-oss-20b, qwen3.8-27b

AI EXTRACTS FIELDS

Java calculates every total

and raises flags when printed figures disagree.

Python FastAPI is the parser / rules service. Groq calls originate in the Java extraction service.

Two AI uses, with different boundaries

Quotation extraction structures a vendor offer. Sahayak helps staff find and understand SUTRA records.

USE CASE 01

Quotation extraction

INPUT	One or more vendor files
MODEL	Groq reads printed fields
OUTPUT	Lines, totals, terms, warnings
NEXT	**Buyer checks before comparison**

USE CASE 02

SUTRA Sahayak

ASK	Hindi / Hinglish / English questions
CONTEXT	Current RFQs, quotations, flags, grades
LINKS	Buttons only for records in context
DRAFT	**Suggests RFQ fields from pasted text**

A draft appears for buyer review. It is saved only after the buyer selects “Create RFQ”.

PROJECT SUTRA

Safety comes from layered checks

Prompt instructions help guide the model, but they are not a complete prompt-injection defence.

1

BOUND THE TASK

Strict JSON schema
Only fields SUTRA expects

2

TREAT FILES AS DATA

Prompt says ignore instructions
inside vendor documents

3

CHECK IN JAVA

Validate fields, match items
Recalculate totals, add flags

4

HUMAN REVIEW

Correct, retry, enter manually
or leave the quotation failed

LIMITATION

OCR and AI can misread values.

Treat low-confidence fields and missing terms as review work.

FAILURE PATH

AUTO can continue to RULES.

When extraction fails: keep the original, retry or correct it by hand.

Prompt injection is reduced by instruction design; no claim of complete resistance.

PROJECT SUTRA

One sign-in token, checked at the edge and by services

Password verification, short-lived claims, active-account checks and backend role enforcement.



sessionStorage | Bearer token on API requests | 15 s active / token-version cache | service tokens expire in 5 min

ADMIN

Users + all admin actions

PROCUREMENT

Buyer workflows

VENDOR

Own portal + own files

APPROVER

Award + audit access

20 requests / minute on login and vendor signup • lockout after 5 failures for 15 minutes

PROJECT SUTRA

Uploads are checked before storage

Validation confirms a file matches an accepted format. It is not antivirus scanning.

NAME
Strip path and control characters

→

TYPE
PDF / XLS / XLSX / DOCX

→

BYTES
Magic-byte signature check

→

PACKAGE
Office ZIP structure check

→

SIZE
20 MB maximum

→

STORE
Original bytes + SHA-256

EDGE CONTROLS

Gateway blocks /internal/**

CORS origins are configured. Login and registration have IP rate limits.

ACCURACY NOTE

SHA-256 is recorded, not used for deduplication.

The current upload path does not perform antivirus / malware scanning.

File validation checks format and structure. It does not prove a document is safe to execute or virus-free.

Financial accuracy stays in the backend

AI reads printed values. Java recalculates. PostgreSQL stores fixed-precision numbers.

CALCULATION PATH

Extracted / corrected lines

→

BigDecimal + HALF_UP

Quantity × unit price, discounts, tax, totals

→

PostgreSQL NUMERIC

Printed totals are compared and mismatches are flagged.

AUDIT TRAIL

Transactional outbox → **Audit service**

sequence + previous hash → **SHA-256 link**

DB trigger blocks updates / deletes on linked entries

More consistency controls

NUMERIC columns • DB constraints • @Version optimistic locking

Tamper-evident and database-guarded, not impossible to bypass.

AI DOES NOT CONTROL FINANCIAL ACCURACY. THE BACKEND DOES.

Testing covers the interface, services and document logic

Three automated suites target different failure points; acceptance checks exercise the assembled workflow.

FRONTEND

Vitest + React Testing Library

- Role-sensitive routes and screens
- Quotation comparison and award forms
- Assistant links and RFQ draft card

10 test files

BACKEND

JUnit + Mockito

- Service and controller rules
- Auth, file, calculation and audit logic
- PostgreSQL service integrations

BUILD SUCCESS

PYTHON

Pytest

- PDF / Excel / Word parsing
- Rule-based quotation extraction
- Matching and FastAPI endpoints

Parser + matcher suites

Acceptance run: all services + the test database + a three-vendor automotive RFQ

PROJECT SUTRA

Recorded test results

Results are shown by suite. Acceptance checks are a separate previously recorded end-to-end run.

52

FRONTEND PASSED

across 10 test files

256

BACKEND TOTAL

0 failures • 0 errors • 1 skipped

38

PYTHON PASSED

pytest suite

75 / 75

ACCEPTANCE

passed • recorded 10 Nov 2026

BACKEND

BUILD SUCCESS

The OCR test was skipped when Tesseract was unavailable.

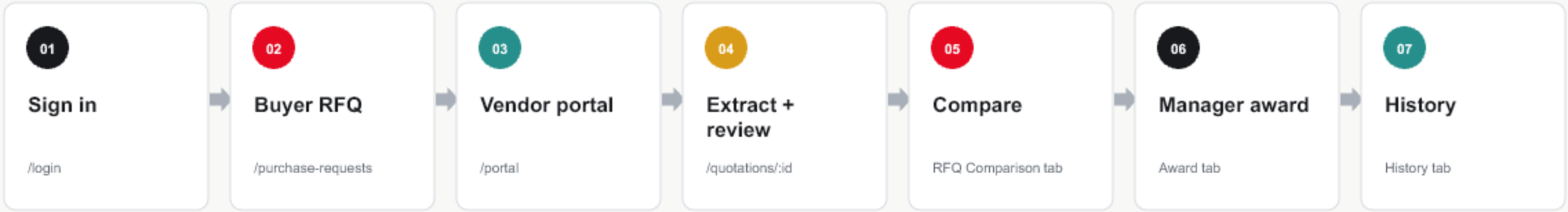
SCOPE NOTE

Acceptance figures are from the recorded 10 Nov run, not a fresh execution during this review. Passing tests do not equal 100% coverage.

PROJECT SUTRA

Live system workflow

Source-based route map for the demo. The running app was unavailable, so this is not a UI screenshot.



Buyer creates request → **Vendor submits files** → **System compares evidence** → **Manager decides**

Suggested live path: show one RFQ with three vendors and use its comparison + history.

ILLUSTRATIVE DIAGRAM • NOT A CAPTURE

PROJECT SUTRA

Five-minute demonstration run

A short, prepared scenario shows each role and the evidence SUTRA records.

0:00–0:40	**Login + dashboard**	Role-specific landing and pending-work badges
0:40–1:20	**Buyer RFQ**	Open a prepared request and review requested items
1:20–2:10	**Vendor portal**	Open the same RFQ and submit prepared PDF / XLSX
2:10–3:15	**Extraction review**	Show parsed lines, calculated totals and flags
3:15–4:10	**Comparison**	Show ranked totals and terms side by side
4:10–5:00	**Award + history**	Manager records the decision; show audit history

Use prepared demo data. Never show credentials, API keys or real vendor-sensitive documents.

Demo dependencies and fallback plan

Know which steps are local and which require an external service.

LOCAL DATA

PostgreSQL 18

RFQs, quotations, files, users

JAVA STACK

Gateway + Spring services

Auth, RFQ, review, compare, approve

DOCUMENTS

Python FastAPI

Parsing, matching, rule-based reader

SCANNED PAGES

PDFBox + Tesseract

Required only for scanned PDF OCR

AI CALL

Groq API

Requires configured key + network

FALLBACK PLAN

Groq unavailable	AUTO can try RULES; assistant unavailable	**Tesseract unavailable**	Use text-based PDF / Excel / Word
Backend unavailable	Use prepared screenshots or acceptance report	**Internet unavailable**	Local parsing remains; Groq calls fail